

In contrast, the diagonal elements of the adjacency matrix of the undirected multigraph *Mgraph_7* also have 1s, indicating that vertices corresponding to such entries have loops. Also, keep in mind that a vertex of an undirected multigraph can have more than one loop, and hence the diagonal entry corresponding to such vertices in the adjacency matrix can have entries greater than 1.

Now, let us print the incidence matrix of the undirected multigraph *Mgraph_7*. To do that, add the following line of code at the end of the program called 'mgraph7.sagews'.

```
print(MG.incidence_matrix( ))
```

Upon execution of the modified program, the following additional output will be displayed on the screen. But how is it different from the incidence matrix of an undirected simple graph?

```
[2 1 1 1 1 0 0 0 0 0 0 0]
[0 1 1 0 0 2 1 1 0 0 0 0]
[0 0 0 0 0 0 1 1 2 1 1 0]
[0 0 0 1 1 0 0 0 0 1 1 2]
```

Well, the incidence matrix of an undirected simple graph only contains 0s or 1s as entries. Furthermore, since every edge has exactly two endpoints, every column of the incidence matrix will have exactly two 1s. However, in the incidence matrix shown above, we also have 2s, indicating the presence of loops in certain vertices in the undirected multigraph *Mgraph_7*. For example, in *Mgraph_7*, columns 1, 6, 9, and 12 correspond to edges that are loops. When there are parallel edges in an undirected multigraph, the columns corresponding to these edges are duplicates of each other. So, in *Mgraph_7*, columns 2 and 3, 4 and 5, 7 and 8, and 10 and 11 are duplicates of each other. Also, notice that the sum of the entries in a column in the incidence matrix is still 2, since an edge only has two endpoints, whether it is a loop, a single edge, or multiple edges (parallel edges).

Now, let us find all the paths between a pair of vertices in an undirected multigraph. Add the following line of code at the end of the program called 'mgraph7.sagews' to find all the paths between vertices 2 and 4 in *Mgraph_7*.

```
print(MG.all_paths(2, 4))
```

Upon execution of the modified program, the following additional output will be displayed on the screen.

```
[[2, 1, 4], [2, 3, 4]]
```

Notice that only distinct paths through different vertices are shown in the output. The method *all_paths()* completely

ignores the presence of loops and multiple edges in the undirected multigraph *Mgraph_7*.

Now, let us test the *longest_cycle()* method on the undirected multigraph *Mgraph_7*. When used on an undirected simple graph, this method gives the length of the longest cycle. Add the following line of code at the end of the program called 'mgraph7.sagews' to test this method on an undirected multigraph.

```
MG.longest_cycle( )
```

Upon execution of the modified program, we get the following error message displayed on the screen. Why do we get this error?

```
ValueError: This method is not known to work on graphs with
multiedges/loops. Perhaps this method can be updated to
handle them, but in the meantime if you want to use it please
disallow multiedges/loops using allow_multiple_edges( ) and
allow_loops( ).
```

Well, this is because the *longest_cycle()* method in SageMath does not support graphs with loops or multiple edges. A workaround for this limitation is to create a new graph without loops and multiple edges from the original graph and then find the longest cycle in this new graph. However, be careful when using this hack because there are multiple definitions for the term 'cycle' in graph theory. I urge you to read the Wikipedia article titled 'Cycle (graph theory)' for further details. Wikipedia considers a *cycle* and a *circuit* as two distinct entities, with every cycle also being a circuit. However, many graph theorists consider a cycle to be equivalent to a circuit (according to the Wikipedia definition), a broader class of graphs. Also, note that the method *longest_cycle()*, whether called as *MG.longest_cycle(induced=True)* or *MG.longest_cycle(induced=False)*, will also lead to an error when used with undirected multigraphs.

Now, let us check whether the undirected multigraph *Mgraph_7* contains an Eulerian circuit. To do that, add the following line of code at the end of the program called 'mgraph7.sagews'.

```
MG.eulerian_circuit(labels=False)
```

Upon execution of the modified program, the following additional output will be displayed on the screen. This clearly indicates that the undirected multigraph *Mgraph_7* has an Eulerian circuit. Please refer to Figure 5 in the previous article and verify that this indeed is an Eulerian cycle.

```
[(1, 4), (4, 4), (4, 3), (3, 3), (3, 2), (2, 2), (2, 3), (3,
4), (4, 1), (1, 2), (2, 1), (1, 1)]
```